How We Built SwiftRoute

Delivery routes that keep working when real traffic hits: the problem, the data, the maths, the app, and how we proved it works.

Real Chicago traffic · April 2026 | OpenStreetMap roads | Google OR-Tools | Streamlit + deck.gl

Track: Operations Research and Decision Intelligence
Problem statement: Last-Mile Delivery Route Optimization
Live demo: swiftroute-routes.streamlit.app
Code: github.com/haricharanparna/swiftroute
Team: [Team name] · [Member names]

CONTENTS

What's inside

1 · The problem and our idea

2 · How the system fits together

3 · The data: real roads and real traffic

4 · The mathematical model

5 · The honest check: replaying the day

6 · The app

7 · Results: scenarios and a 10-day validation

8 · Testing and reproducibility

9 · Challenges, lessons and limitations

10 · Tech stack and project files

10/10
test days cheaper *and* more on time

−32.6%
average daily cost

+23.8
points more deliveries on time

2.66 M
real traffic readings used

CHAPTER 1

1 The problem and our idea

Last-mile delivery, from the depot to the customer's door, is the most expensive leg of shipping. Big companies have routing software; most small fleets still plan by hand, usually by driving to the **nearest stop next**.

That rule ignores the three things that decide whether a package arrives on time:

📦 Capacity
Each van can only carry so many packages.

🕑 Time slots
Customers are only home inside their delivery window.

🚦 Traffic
Roads slow down at different hours, worst on weekday afternoons.

The official problem statement asked for exactly this: *"Optimize last-mile delivery routes under capacity, time-window and traffic constraints to reduce cost and improve service"*, with a routing engine, a visualization of routes and KPIs, and a scenario analysis.

> **Our idea in one line:** plan every van around capacity, time slots and **real hourly traffic**, then **replay the whole day against real traffic** to prove the plan holds, instead of trusting it.

Who it's for

Dispatchers at small and medium fleets (florists, pharmacies, grocers, parcel couriers) who need a plan in seconds, drivers who need clear directions, and managers who need to answer "what if" questions about trucks and demand.

CHAPTER 2

2 How the system fits together

SwiftRoute is a Python pipeline with a web app on top. Data flows left to right:

1 · Stops
Typed, spoken, or "my location" → map points (OpenStreetMap)

→

2 · Roads
Real distance and empty-road drive time for every pair (OSRM)

→

3 · Expected traffic
Each drive slowed by Chicago's factor for its hour

→

4 · Optimize
OR-Tools: capacity, time slots, lowest cost

→

5 · Actual replay
Minute-by-minute with real traffic; all numbers come from here

The same roads, traffic and replay are applied to the **"nearest stop next" benchmark**, a deliberately simple stand-in for hand-planning, so every comparison is like-for-like.

Main modules

File	What it does
`traffic.py`	Downloads Chicago Traffic Tracker readings and builds an hourly slowdown profile
`geo.py`	Geocoding, road distance/time tables (cached), turn-by-turn routes
`solver.py`	OR-Tools optimizer, the nearest-stop benchmark, KPI calculation
`plan.py`	Ties the pipeline together and runs the hour-by-hour replay
`app.py`	Streamlit app: live plan, replay, driver directions, what-if, traffic, about
`replay.py`	Animated "replay the day" map (deck.gl, bundled locally)
`i18n.py`	All text in English, Spanish and French, including driving directions
`scenarios.py`, `validate.py`	Scenario analysis and the 10-day robustness test
`tests/`	11 automated correctness tests

CHAPTER 3

3 The data: real roads and real traffic

Traffic: City of Chicago Traffic Tracker

We used the city's open dataset *Historical Congestion Estimates by Segment, 2024–2026* (`4g9f-3jbs`): speed estimates for ~1,000 arterial road segments roughly every 10 minutes, derived from GPS on CTA buses.

2,664,325
speed readings

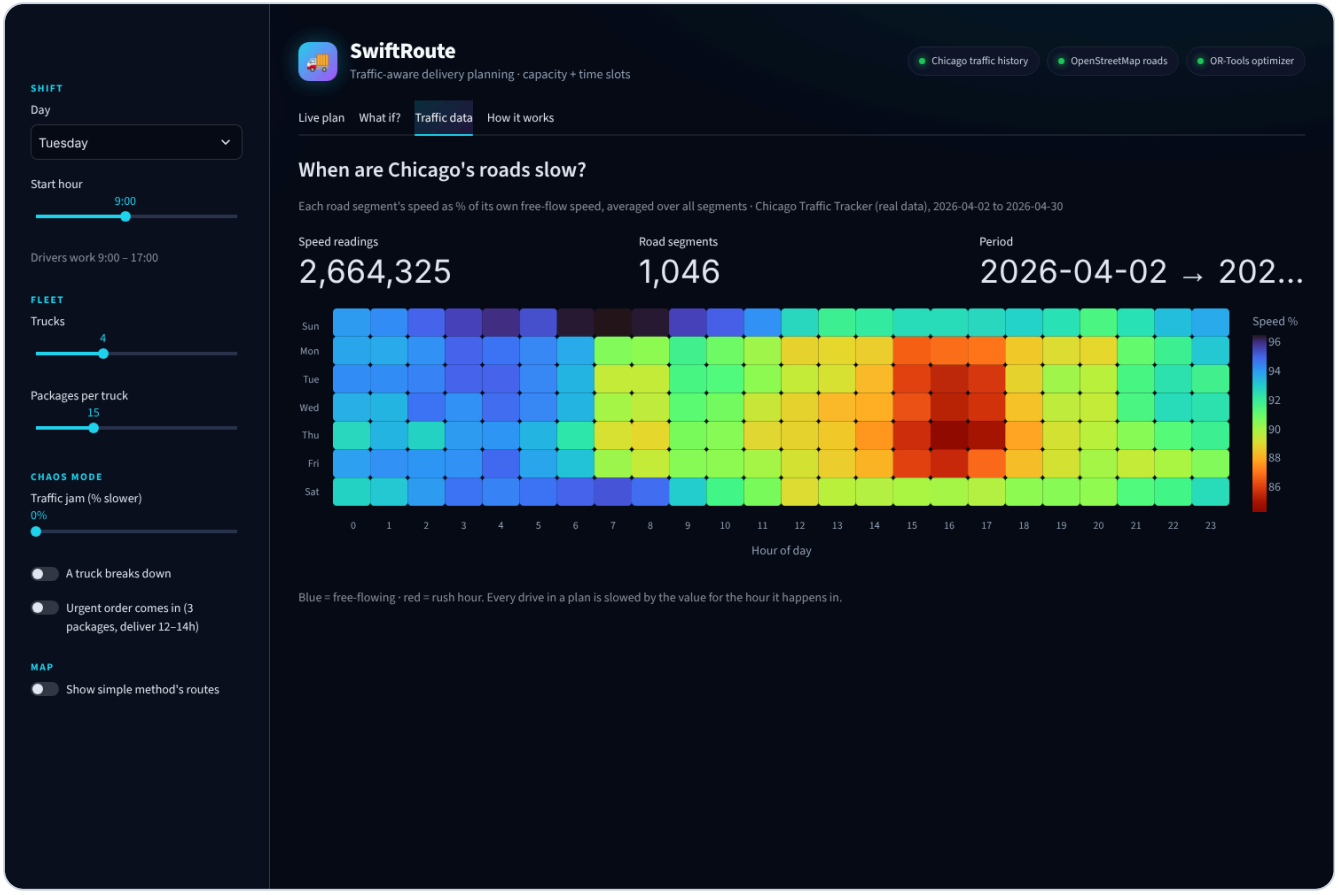
1,046
road segments

4 weeks
2026-04-02 → 2026-04-30

Method

1. Average speed for every (segment, weekday, hour).
2. Each segment's **free-flow speed** = its 90th-percentile hourly speed.
3. **Speed factor** = hourly speed ÷ free-flow speed (1.0 = flowing freely).
4. Average the factor across all segments → one value per hour of the week (168 values).

Comparing each road with *itself* matters: a highway at 40 mph and a side street at 15 mph can both be "free-flowing".



The resulting profile: weekday afternoons (3–6 pm) are the slowest, around 85% of free-flow speed.

Roads: OpenStreetMap via OSRM

For every pair of stops we fetch the real road distance and the empty-road drive time (which already knows highways are faster than side streets). Tables are cached in the repo, so experiments always use identical road data.

CHAPTER 4

4 The mathematical model

This is a classic operations research problem: the **Vehicle Routing Problem with Capacity and Time Windows** (CVRPTW), extended with hour-dependent travel times.

Objective: minimize the real daily cost

$$\text{minimize} \quad 0.60 \cdot \Sigma_{\text{legs}}\, d_{ij} \;+\; 25 \cdot \Sigma_{\text{vans}}\, T_k \;+\; 10{,}000 \cdot (\text{undelivered orders}) \quad [+ \text{ optional fixed cost per van used}]$$

d_{ij} km driven on each leg · T_k hours each van is out (leave → return)

Subject to

each order served at most once

Σ packages on van $k \le Q$ **capacity**

$\text{open}_i \le t_i \le \text{close}_i$ **time window**

$t_j \ge t_i + 5\ \text{min} + \tau_{ij}(h)$ **time flow**

$\tau_{ij}(h) = \text{free-flow time}_{ij} \div f(\text{day}, h)$ **traffic**

The $10,000 penalty means an order is only dropped when no van can reach it inside its slot; dropped orders are shown in red and counted against every KPI, never hidden.

Solving it

- **Google OR-Tools** routing solver: a "path cheapest arc" first solution, improved by **guided local search**.
- Interactive app: stops after a few seconds. Experiments: stop after a fixed **1,000 solutions**, so results are identical on any computer.
- It is a strong heuristic, **not a proof of optimality**; more search finds slightly better routes.

Assumptions

$0.60/km fuel and wear · $25/hour per driver · 5 minutes per delivery · 0.25 kg CO_2/km · 8-hour shift.

CHAPTER 5

5 The honest check: replaying the day

A plan can look perfect on paper and still be late in real traffic. So we use traffic twice.

Planning with *expected* traffic

The optimizer can't know exactly when each drive will happen, so a drive from stop A uses the traffic factor of the hour we **expect** to leave A (the middle of A's time slot; the shift start for the depot).

Checking with *actual* traffic

Every route is then **replayed minute by minute**: each drive is slowed by the traffic of the hour it **actually** starts in; vans wait if they arrive before a slot opens.

> **All reported numbers come from the replay**: cost, on-time %, deliveries, km, driver hours. If traffic makes a planned delivery late, it shows up as late.

The benchmark

"Nearest stop next": from the current position, go to the nearest unvisited stop that still fits in the van. It respects capacity but doesn't plan around closing times; its lateness is measured in the same replay. It is a deliberately simple stand-in for hand-planning, **not a competing optimizer**.

How we score both methods

KPI	Definition
Cost	fuel per km + driver pay per hour out (+ optional cost per van used)
Delivered	orders reached inside their time slot by the plan
On time %	on-time deliveries ÷ **all** orders, so undelivered orders count against the method
Distance, driver hours, CO_2	from the replayed routes

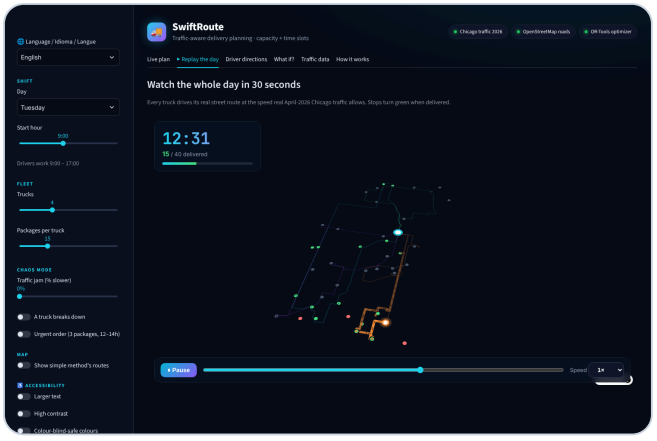
CHAPTER 6

6 The app

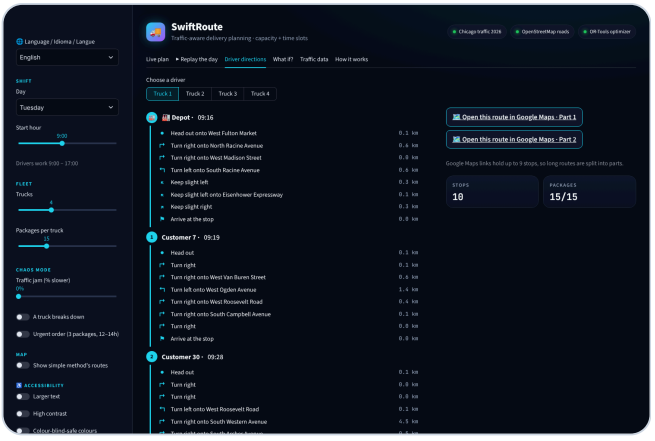
A Streamlit web app with six tabs. It runs on a laptop or for free on Streamlit Community Cloud.



Live plan. Savings vs. the benchmark, KPI cards, routes on real Chicago streets, fleet panel, chaos mode (traffic jam, van breakdown, urgent order) and a "show all routes" home button.

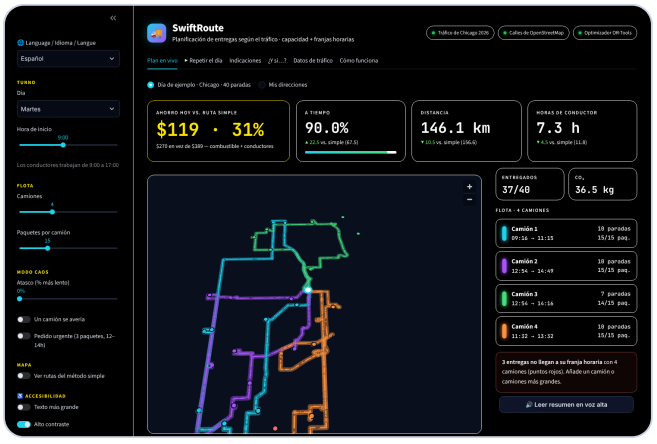


Replay the day. Every van drives its real route at real traffic speed; a clock runs and stops turn green when delivered. Built with deck.gl, bundled locally.

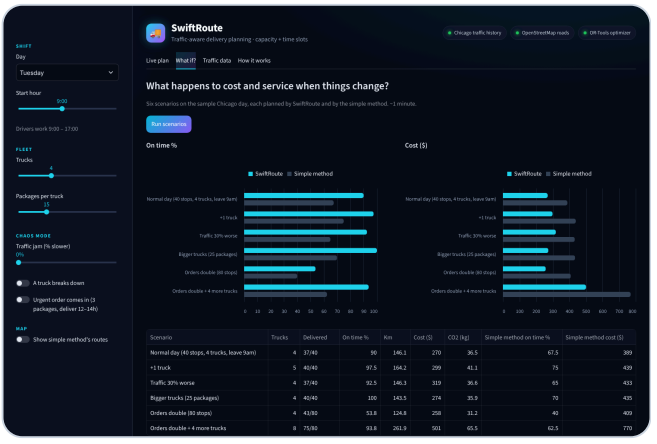


Driver directions. Turn-by-turn steps in three languages, plus one tap to open the route in Google Maps.

Built for everyone



Spanish with high contrast on.



What if? scenario charts.

Input

Type addresses, **speak** them (speech-to-text in the chosen language), or use **"my location"** for the depot. Addresses are found with OpenStreetMap.

Accessibility

Larger text, high contrast, colour-blind-safe (Okabe-Ito) colours, reduce motion, read-aloud summary, keyboard focus outlines, screen-reader map descriptions.

Languages and themes

English, Español, Français (the whole app, including directions); light and dark mode; an animated intro screen.

Exports

Download every driver's schedule and the scenario results as CSV; open any route in Google Maps.

CHAPTER 7

7 Results

Robustness: 10 different random days

40 orders per day, 4 vans × 15 packages, Tuesday 9:00–17:00, real roads and April-2026 traffic (`validate.py`).

10/10
days cheaper and more on time

−32.6%
cost (−15.7% to −51.1%)

+23.8
points on time (+12.5 to +32.5)

36.5
of 40 delivered (benchmark 35.2)

Day	SwiftRoute cost	Benchmark cost	Saving	Delivered	Benchmark delivered	On time	Benchmark on time
1	$269	$471	**42.9%**	36/40	33/40	87.5%	62.5%
2	$287	$382	**24.9%**	37/40	37/40	92.5%	75.0%
3	$233	$336	**30.7%**	35/40	32/40	87.5%	60.0%
4	$274	$560	**51.1%**	37/40	36/40	92.5%	70.0%
5	$301	$425	**29.2%**	37/40	36/40	92.5%	67.5%
6	$229	$409	**44.0%**	33/40	32/40	82.5%	60.0%
7	$270	$389	**30.6%**	37/40	35/40	90.0%	67.5%
8	$295	$420	**29.8%**	38/40	38/40	95.0%	82.5%
9	$300	$412	**27.2%**	40/40	40/40	100.0%	67.5%
10	$257	$305	**15.7%**	35/40	33/40	87.5%	57.5%

SwiftRoute delivered at least as many orders as the benchmark on every day, so the savings don't come from skipping work.

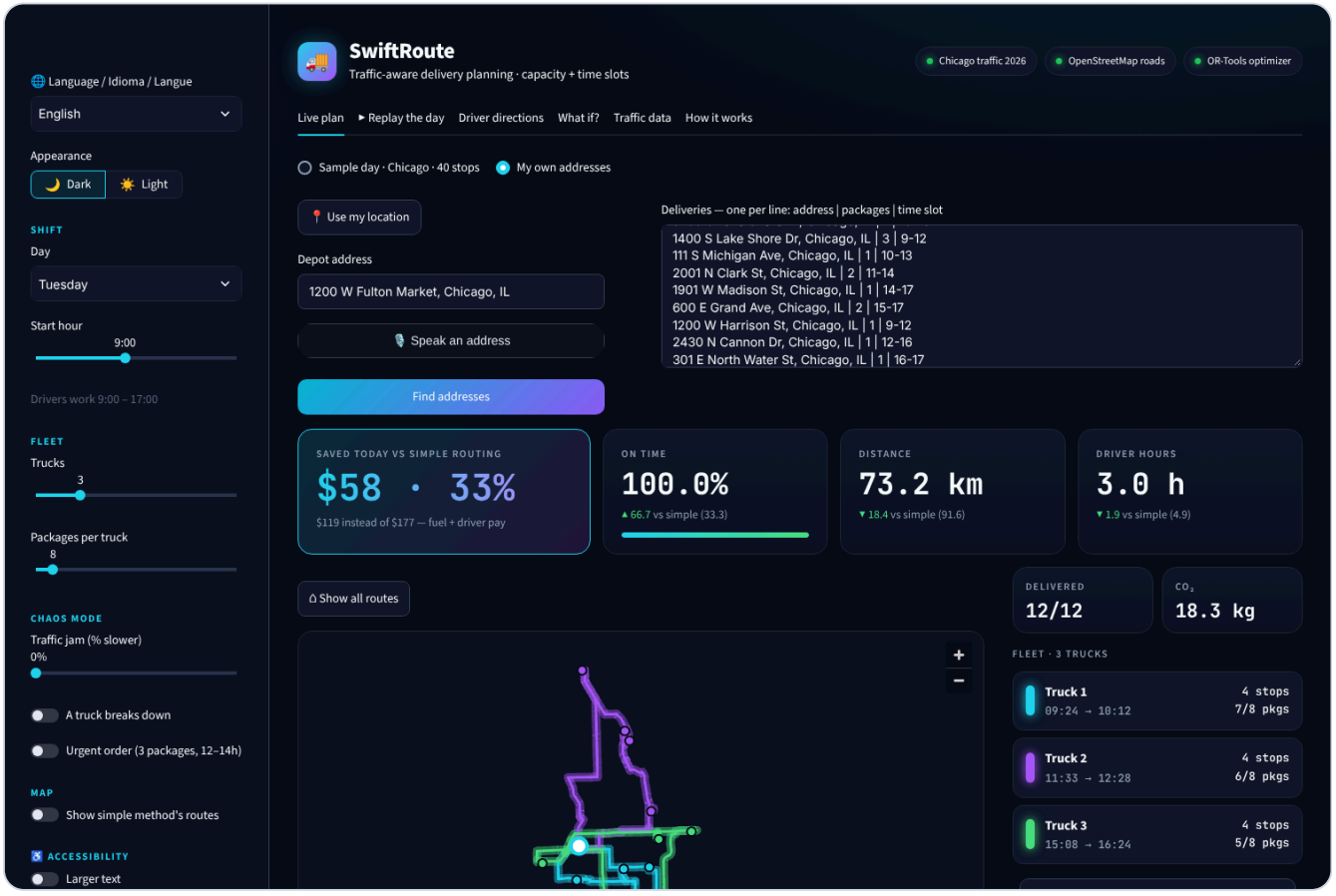
Scenario analysis (sample Chicago day)

Scenario	Delivered	On time	Cost	Bench. delivered	Bench. on time	Bench. cost
Normal day (40 stops, 4 trucks, leave 9am)	37/40	90.0%	$270	35/40	67.5%	$389
+1 truck	40/40	100.0%	$294	40/40	75.0%	$439
Traffic 30% worse	37/40	92.5%	$319	35/40	65.0%	$433
Bigger trucks (25 packages)	**40/40**	**100.0%**	**$274**	**40/40**	**70.0%**	**$435**
Orders double (80 stops)	43/80	52.5%	$252	40/80	40.0%	$409
Orders double + 4 more trucks	75/80	93.8%	$488	72/80	62.5%	$770

> **Insight:** the fleet is limited by **van size** more than by time. Bigger vans reach 40/40 delivered and 100% on time at almost the same cost, a decision a manager can act on.

End-to-end test: a florist's real day

We entered a realistic day through the app in a real browser, exactly as a user would: **"Fulton Flowers"**, 3 vans × 8 bouquets, 18 bouquets to 12 real Chicago addresses (Willis Tower, Field Museum, Wrigley Field, Navy Pier, a far stop in Hyde Park, a tight 4–5 pm slot). Then we downloaded the plan and checked it independently against the original orders.



12/12 delivered, 100% on time (benchmark 33%), $119 vs $177, vans carrying 7, 6 and 5 bouquets.

- ✔ All 12 addresses found and visited exactly once
- ✔ No van over 8 bouquets; package counts match the orders
- ✔ Every arrival inside its slot (e.g. Willis Tower 9:39 in 9–11; North Water St 16:01 in 16–17)
- ✔ Every van starts and ends at the depot, with times only moving forward

19 / 19 checks passed. The test also taught us something: three vans were needed because each van makes one trip (no reloading), which led to the "cost per van" option and our top next step, multi-trip routing.

CHAPTER 8

8 Testing and reproducibility

Reproducible by design

- ✔ Fixed random seeds for every test day
- ✔ Traffic profile committed to the repo (`data/traffic_profile.csv`)
- ✔ Road tables cached in the repo (`data/road_cache/`)
- ✔ Solver stops after a fixed 1,000 solutions, not a number of seconds

We ran the full 10-day validation twice: the outputs were **byte-for-byte identical**.

```
python validate.py          # 10-day test   -> results/validation.csv
python scenarios.py         # 6 scenarios   -> results/scenarios.csv
python -m pytest tests -q   # 11 correctness tests
```

11 automated tests

- ✔ No stop visited twice; routes start/end at depot
- ✔ Capacity never exceeded
- ✔ Planned arrivals inside time windows
- ✔ An order too big for any van is left unassigned, no crash
- ✔ Unassigned orders count against on-time %
- ✔ "No solution" returns an empty plan, not an error
- ✔ Same search budget → identical answer
- ✔ Benchmark respects capacity
- ✔ Replay catches lateness the plan missed
- ✔ Orders outside the shift reported as undelivered
- ✔ Cost per van never uses more vans and is charged correctly

CHAPTER 9

9 Challenges, lessons and limitations

Challenges we ran into

- **Results that changed between runs.** Our first validation gave slightly different numbers each time, because the solver stopped after a number of seconds. A fixed solution budget and cached road data fixed it.
- **Traffic data at scale.** The city's servers timed out on years of history; we download only the needed columns in weekly chunks and aggregate locally (2.66 M readings in ~90 seconds).
- **Trusting the plan.** A plan can be late in real traffic, which is why we built the replay and report only replayed numbers.
- **Bugs found by testing:** a map zoom formula off by a factor of two (deck.gl uses 512-pixel tiles), a dot-size setting that painted over the map, and our own wrong first explanation for why the florist needed three vans.

What we learned

- Real data beats assumptions: Chicago's worst traffic is 3–6 pm on weekdays, not the morning.
- A fair comparison needs the same inputs, the same scoring and honest reporting of misses.
- Optimization only helps if people can use it, hence directions, languages and accessibility.

Honest limitations

! Demo customers are generated (fixed seeds), not real orders.

! Traffic is one city-wide factor per hour (arterial streets, bus-GPS estimates), not per street.

! Each van makes one trip; it can't return to the depot to reload mid-day.

! SwiftRoute sometimes drives more km, trading distance for fewer driver hours and late deliveries.

! Uses free public OSRM / Nominatim / speech servers; production would self-host them.

What's next

Multi-trip routing (reloading) · live traffic feeds and more cities · a driver phone app with live re-routing · real order data from partner fleets.

CHAPTER 10

10 Tech stack

Area	Tools
Language	Python 3.13
Optimization	Google OR-Tools (vehicle routing, guided local search)
Data	pandas · City of Chicago open data (Socrata API) · OpenStreetMap
Routing services	OSRM (distances, drive times, turn-by-turn) · Nominatim (geocoding)
App	Streamlit · streamlit-mic-recorder (voice) · streamlit-js-eval (location)
Maps and charts	pydeck / deck.gl · MapLibre · Altair
Testing	pytest · Playwright (browser end-to-end test)
Hosting and code	Streamlit Community Cloud · GitHub

Run it yourself

```
git clone https://github.com/haricharanparna/swiftroute
cd swiftroute
pip install -r requirements.txt
streamlit run app.py
```

Team contributions

Member	Contribution
[Name]	[What they built]
[Name]	[What they built]
[Name]	[What they built]

Data: City of Chicago Traffic Tracker (open data) · © OpenStreetMap contributors.